Program 1:Working with Maven: Creating a Maven Project, Understanding the POM File

Dependency Management and Plugins

Step 1: Create a Maven project by running following command into terminal:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp - DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
brijesh@Ubuntu:~$ cd devops
brijesh@Ubuntu:~/devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 MB at 5.8 MB/s)
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/brijesh/devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/brijesh/devops/MyMavenApp
[INFO] ----------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------------
[INFO] Total time:  7.977 s
[INFO] Finished at: 2026-03-21T12:26:31+05:30
[INFO] ----------------------------------------------------------------------------
brijesh@Ubuntu:~/devops$
```

This command generates a basic Maven project structure with a sample App.java and AppTest.java.

```
brijesh@Ubuntu:~/Devops/MyMavenApp$ tree
.
├── pom.xml
├── src
│   ├── main
│   │   └── java
│   │       └── com
│   │           └── example
│   │               ├── App.java
│   │               └── MainClass.java
│   └── test
│       └── java
│           └── com
│               └── example
│                   └── AppTest.java
└── target
    ├── classes
    │   └── com
    │       └── example
    │           ├── App.class
    │           └── MainClass.class
    ├── generated-sources
    │   └── annotations
    ├── generated-test-sources
    │   └── test-annotations
    ├── maven-archiver
    │   └── pom.properties
    ├── maven-status
    │   └── maven-compiler-plugin
    │       ├── compile
    │       │   └── default-compile
    │       │       ├── createdFiles.lst
    │       │       └── inputFiles.lst
    │       └── testCompile
    │           └── default-testCompile
    │               ├── createdFiles.lst
    │               └── inputFiles.lst
    ├── MyMavenApp-1.0-SNAPSHOT.jar
    ├── surefire-reports
    │   ├── com.example.AppTest.txt
    │   └── TEST-com.example.AppTest.xml
    └── test-classes
        └── com
            └── example
```

This “Build Success” will confirm that the project is created successfully

Step 2: Go to the project directory using command cd MyMavenApp

Step 3: Open and edit pom. xml file using command gedit pom.xml

- Go to https://mvnrepository.com
  - Search for a dependency “junit”
  - Select the latest version and copy the dependency XML snippet.
  - Paste it under the <dependencies> section in pom.xml
  - Updated pom.xml:

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenApp</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>


  <name>MyMavenApp</name>
  <url>http://maven.apache.org</url>

  <!-- Properties: Customize Java version or plugin versions -->
<properties>
<maven.compiler.source>11</maven.compiler.source>
<maven.compiler.target>11</maven.compiler.target>
</properties>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <!-- Build: Configuring plugins and build settings -->
<build>
  <plugins>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.8.1</version>
    </plugin>

    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.1.0</version>
      <configuration>
        <mainClass>com.example.MainClass</mainClass>
      </configuration>
    </plugin>
```

Update App.java:

```java
package com.example;

/**
 * Hello world!
 *
 */
public class App
{
    public static void main( String[] args )
    {
        int num1 = 10;
        int num2 = 20;

        int sum = num1 + num2;

        System.out.println("Sum = " + sum);
    }
}
```

Step 4: Build the project using command mvn compile

The project building will start like this.

```
brijesh@Ubuntu:~/Devops/MyMavenApp$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----------------------< com.example:MyMavenApp >-----------------------
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/brijesh/Devops/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 2 source files to /home/brijesh/Devops/MyMavenApp/target/classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.792 s
[INFO] Finished at: 2026-03-21T12:42:42+05:30
[INFO] ------------------------------------------------------------------------
brijesh@Ubuntu:~/Devops/MyMavenApp$
```

The “Build Success” message ensures that our Maven project is built successfully.

Step 5: Package the project using command mvn package.

Step 6: Run the test on project using command mvn test.

Step 7: To run the Maven application

- Go to target directory present in MyMavenApp using command cd target
  - Execute the JAR file with the command:

  java -jar MyMavenApp-1.0-SNAPSHOT.jar

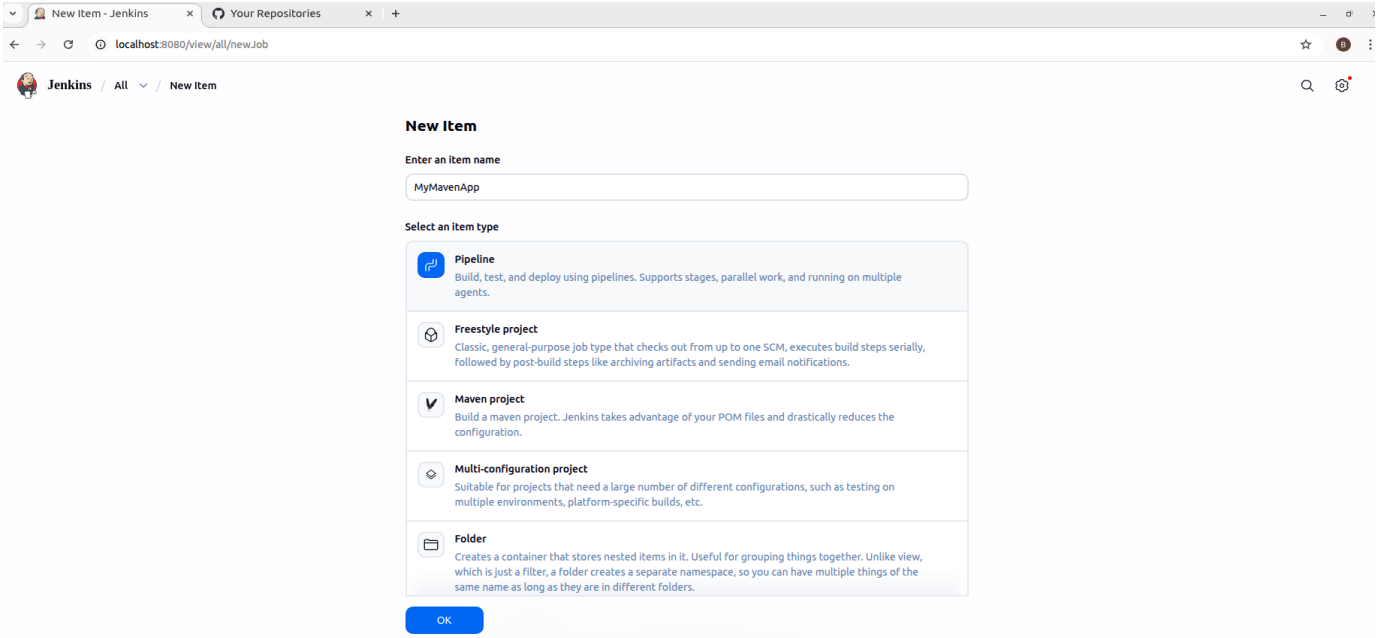
  - After completing the execution, the output will look like:

```
└── target
    ├── classes
    │   └── com
    │       └── example
    │           ├── App.class
    │           └── MainClass.class
    ├── generated-sources
    │   └── annotations
    ├── generated-test-sources
    │   └── test-annotations
    ├── maven-archiver
    │   └── pom.properties
    ├── maven-status
    │   └── maven-compiler-plugin
    │       ├── compile
    │       │   └── default-compile
    │       │       ├── createdFiles.lst
    │       │       └── inputFiles.lst
    │       └── testCompile
    │           └── default-testCompile
    │               ├── createdFiles.lst
    │               └── inputFiles.lst
    ├── MyMavenApp-1.0-SNAPSHOT.jar
    ├── surefire-reports
    │   ├── com.example.AppTest.txt
    │   └── TEST-com.example.AppTest.xml
    └── test-classes
        └── com
            └── example
                └── AppTest.class

28 directories, 15 files
brijesh@Ubuntu:~/Devops/MyMavenApp$ cd target
brijesh@Ubuntu:~/Devops/MyMavenApp/target$ java -jar MyMavenApp-1.0-SNAPSHOT.jar
Rio is the Goat
brijesh@Ubuntu:~/Devops/MyMavenApp/target$
```

This will show output as the code present in App.java file we can edit the text to get any other output.

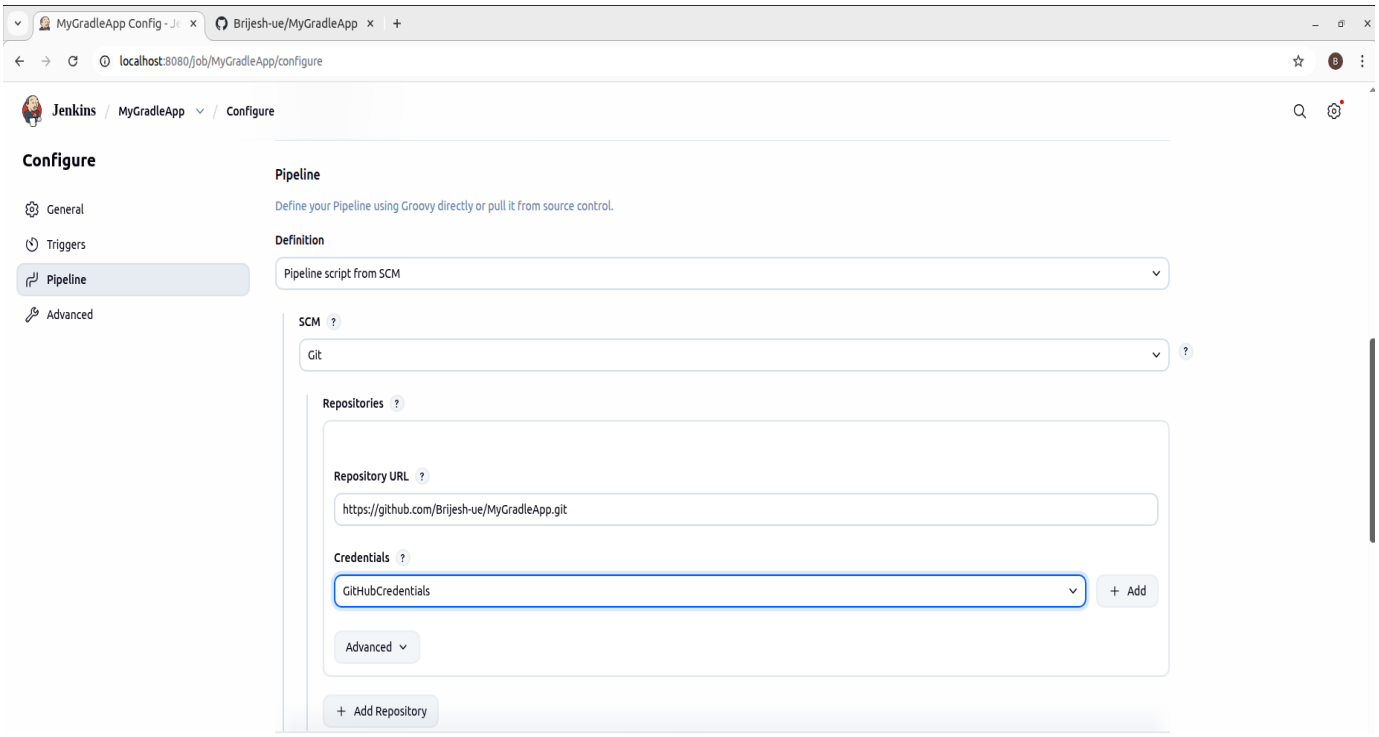
Step 1: Create a New Pipeline Job

Go to Dashboard → + New Item.
Enter name, select Pipeline, and click OK.



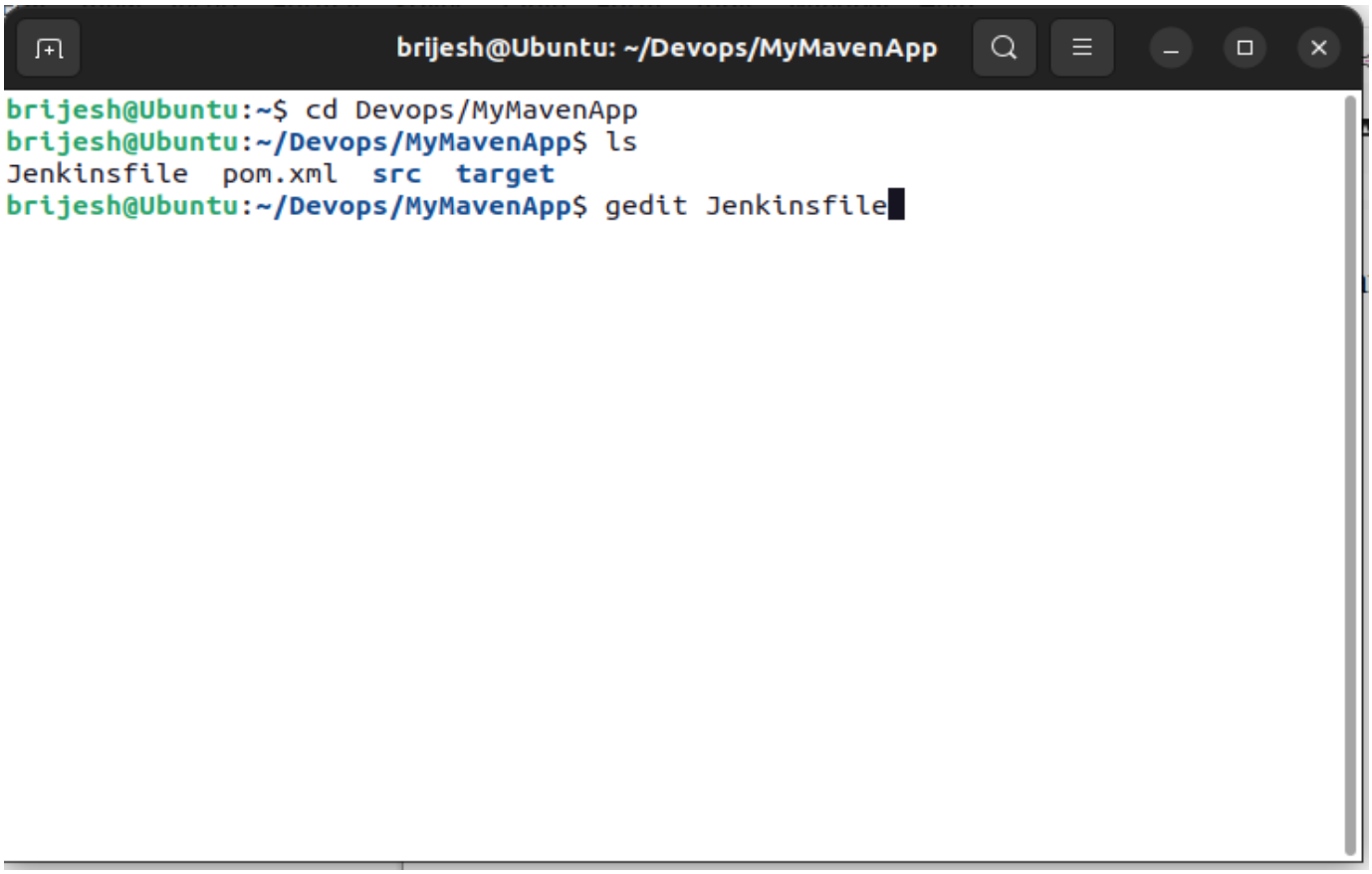
Step 2: Configure Pipeline from SCM

Scroll down and select Pipeline script from SCM.
SCM: Git Enter repository URL and select GitHub credentials.
Click Apply and Save.



Step 3: Create Jenkinsfile

Create a Jenkinsfile in your MyMavenApp project and add the pipeline script.



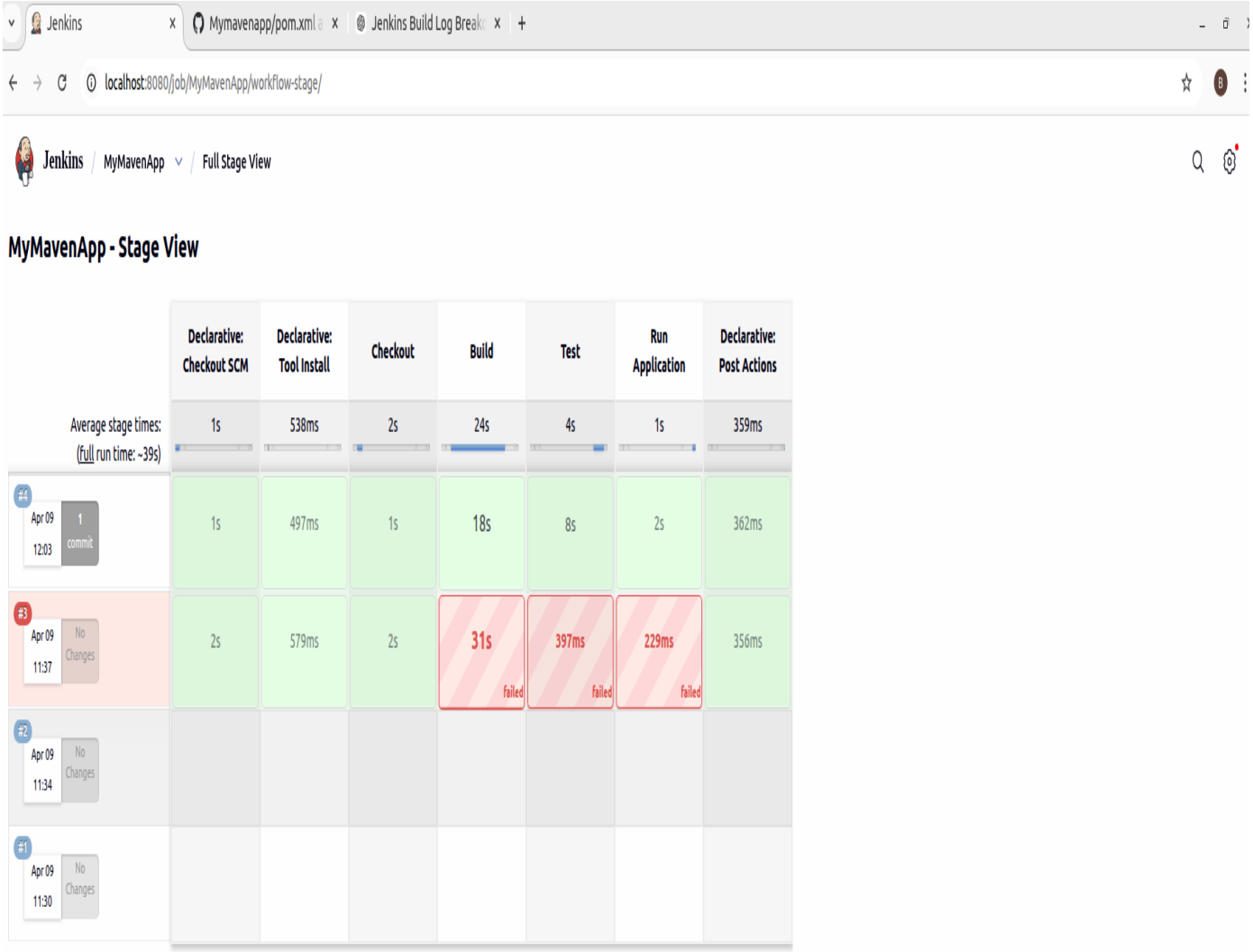
Enter following code into jenkins file

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url:
'https://github.com/Brijesh-ue/Mymavenapp.git'
            }
        }
```

```
        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'  // Run unit tests
            }
        }




        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
            }
        }


    }

    post {
        success {
            echo 'Build and deployment successful!'
        }
        failure {
            echo 'Build failed!'
        }
    }
}
```

Step 4: Update pom.xml

Change source and target versions from 1.7 to 1.8.
Commit changes and push to your github repository.

Step 5: Build the Pipeline

Go to Jenkins and click Build Now.
Monitor Build Stages Use Stage View and Full Stage View to monitor execution.